

El modelo no lo sabe todo.

M2 cerró con su harness y el scorecard del baseline. M3 arranca con la pregunta siguiente: ¿cómo le damos al modelo conocimiento que no tiene — el de SU dominio, el de ayer — sin reentrenarlo? La respuesta es RAG: buscar primero, responder después.

[PIPELINE RAG](#)[CHUNKING Y EMBEDDINGS](#)[SU PRIMER RAG](#)

La sesión de hoy.



Recap: qué tienen y qué les falta



Por qué RAG: las cuatro razones



El pipeline completo, de ingest a generate



Chunking: el arte de partir documentos



Embeddings y bases vectoriales



Lab A



Su primer RAG ingenuo • tarea: harness al RAG

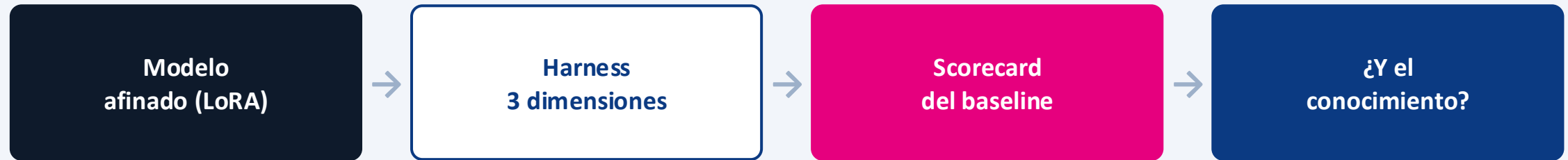


Lab B

Su modelo no sabe lo de ayer.

Knowledge cutoff, datos privados, costos y trazabilidad: por qué el fine-tuning no alcanza.

Lo que ya tienen.



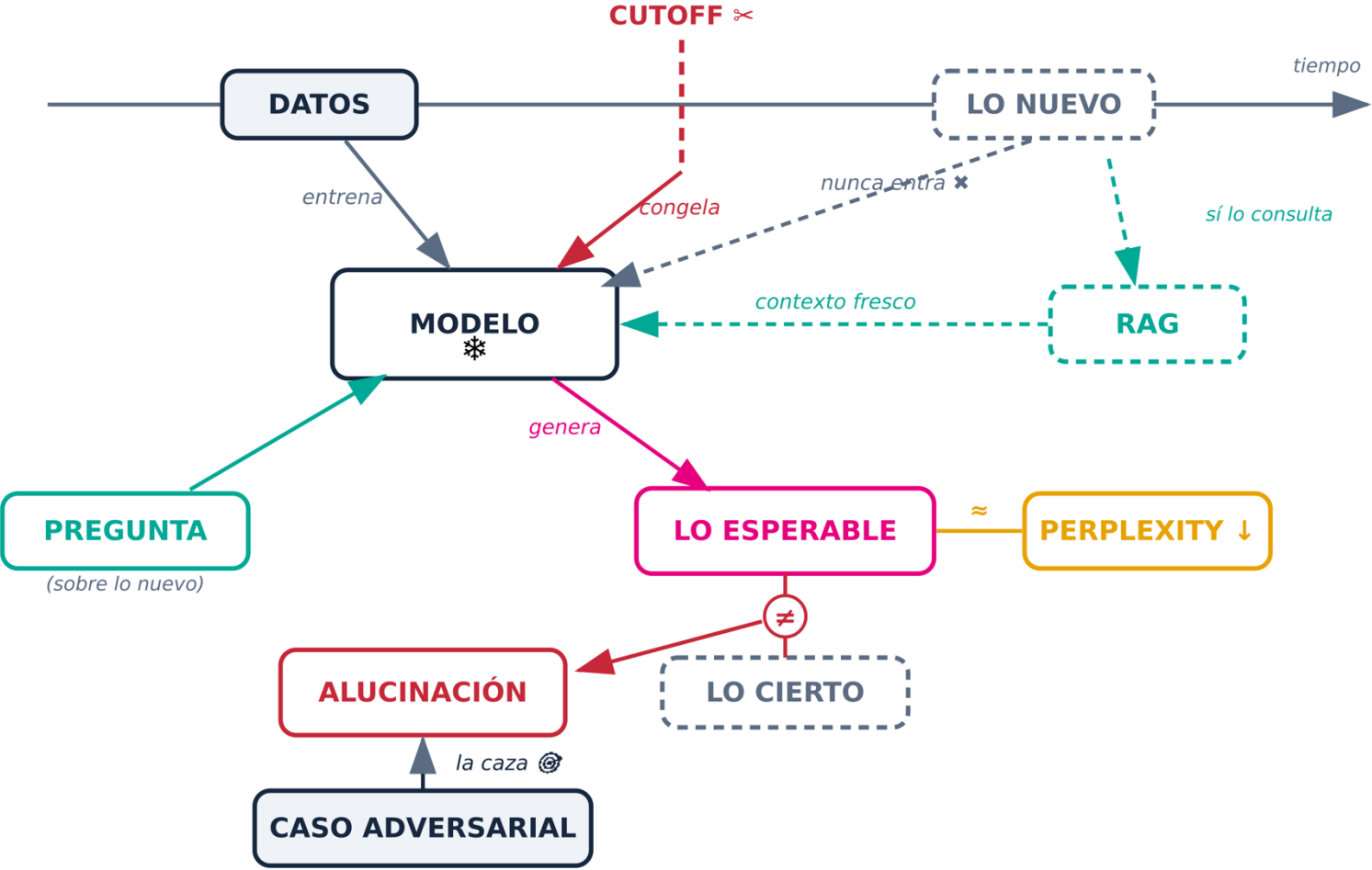
Tienen un modelo que habla como su dominio y una vara para medirlo. Pero el scorecard ya les mostró el hueco: cuando la respuesta exige **datos concretos y actualizados** — la ficha de ESTE producto, la norma vigente, el protocolo de ESTA entidad — el modelo inventa o se queda corto.

Hoy no cambiamos el modelo: le damos una **biblioteca y la instrucción de consultarla antes de responder.**

Pregúntele a su modelo por **la norma que salió ayer**.

¿Qué hace? No dice "no sé". **Responde igual, con total seguridad** — con lo que había en su entrenamiento, o con algo inventado que suena perfecto.

El modelo tiene un **knowledge cutoff**: su conocimiento quedó congelado el día que terminó su entrenamiento. Todo lo posterior — y todo lo privado de su organización — simplemente no existe para él. Y en el Lab de S06 ya lo midieron: el caso adversarial de alucinación es exactamente este fallo.



Cuatro razones para no reentrenar.

KNOWLEDGE CUTOFF

Lo nuevo no está

El mundo cambia todos los días; los pesos del modelo, no. Reentrenar por cada novedad es imposible; un índice de documentos se actualiza en minutos.

DATOS PRIVADOS

Lo suyo tampoco

El catálogo, los protocolos internos, la normativa local: nada de eso estaba en el corpus público de entrenamiento. RAG los pone a disposición sin exponerlos a un tercero.

COSTO

Consultar es más barato

Fine-tuning: horas de GPU por cada actualización. RAG: indexar el documento nuevo y listo. El conocimiento vive en la base, no en los pesos.

TRAZABILIDAD

Poder citar la fuente

"Según el documento X, página Y". Un sistema que cita de dónde sacó la respuesta se puede auditar y corregir — clave en legal, salud y todo dominio serio.

¿Fine-tuning o RAG? Preguntas distintas.

Fine-tuning (M1)

enseña FORMA y habilidad

el estilo del dominio, el formato de salida, el tono, la tarea

RAG (M3)

da CONOCIMIENTO consultable

hechos actualizables, datos privados, fuentes citables

Juntos

lo que ustedes tendrán

un modelo que habla como su dominio Y consulta los datos de su dominio

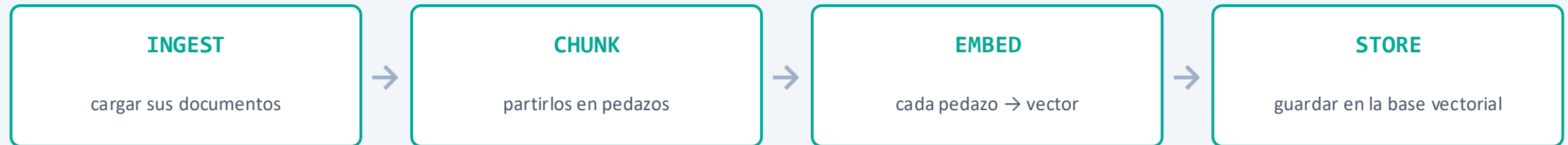
La pregunta correcta nunca es "¿cuál es mejor?" sino "¿mi problema es de habilidad o de conocimiento?". Si la respuesta cambia con el tiempo o vive en SUS documentos → RAG. Si es un patrón estable de comportamiento → fine-tuning. Casi siempre: ambos.

Buscar, luego responder.

Las siete etapas del pipeline — y las dos fases en que se agrupan.

Siete pasos, dos fases.

FASE 1 · INDEXACIÓN (offline, una vez por corpus)



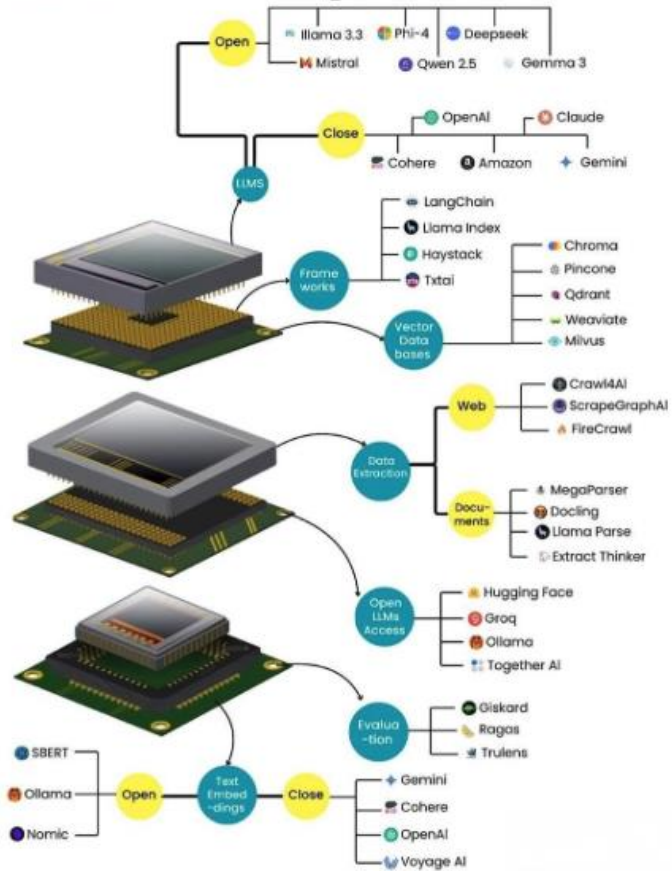
FASE 2 · CONSULTA (online, en cada pregunta)



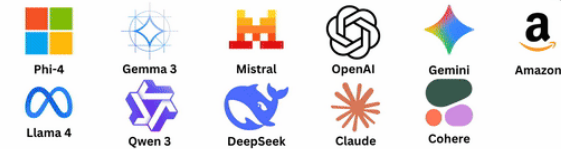
La fase 1 se paga UNA vez y se repite solo cuando el corpus cambia. La fase 2 corre en cada pregunta del usuario — por eso su latencia y su calidad son las que el usuario siente.

En el lab de hoy implementan las siete – sin frameworks, para entender cada pieza. LangChain/LlamaIndex vienen después.

RAG Developer's Stack



LLMs



Frameworks



Vector Databases



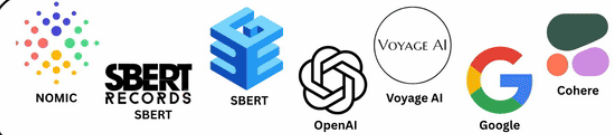
Data Extraction



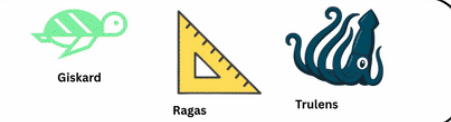
Open LLMs Access



Text Embeddings



Evaluation



Rahul Agarwal
Follow for More

Anatomía del prompt aumentado.

INSTRUCCIÓN

"Responde SOLO con base en el contexto proporcionado."

La regla del juego: el modelo no debe usar su memoria, sino la biblioteca.

CONTEXTO

Los k chunks recuperados, cada uno con su fuente.

Aquí entra lo que la búsqueda encontró. Con fuente: eso habilita la cita.

PREGUNTA

La consulta original del usuario, tal cual.

Al final, para que quede fresca en la atención del modelo.

VÁLVULA DE ESCAPE

"Si la respuesta no está en el contexto, di que no la tienes."

La línea anti-alucinación: le da permiso explícito de decir "no sé".

La válvula de escape es la línea más importante del prompt: sin ella, el RAG alucina igual que el modelo solo.

RAG reduce la alucinación. No la elimina.

La palabra técnica es **grounding**: anclar la generación a evidencia recuperada. En vez de pedirle al modelo que recuerde, le pedimos que lea y sintetice — una tarea donde los LLM son mucho más confiables. Y como el contexto trae fuente, la respuesta se puede auditar.

Pero un RAG puede fallar aunque el pipeline esté bien montado — y lo van a ver en el lab:

- Si la búsqueda trae el chunk equivocado, el modelo sintetiza — con toda seguridad — a partir de evidencia irrelevante.
- Si el chunk correcto llega pero incompleto, la respuesta sale a medias: el chunking decidió por ustedes.
- Y si el modelo ignora el contexto, responde de memoria igual que antes — por eso la válvula de escape y por eso se evalúa.

El arte de partir.

El chunk es la unidad de recuperación: cómo partan sus documentos define qué puede encontrar su RAG.

Por qué no meter el documento entero.

RAZÓN 1 • CONTEXTO

El prompt no es infinito

La ventana de contexto del modelo tiene límite — y aunque no lo tuviera: más texto irrelevante en el prompt = más ruido y más costo por consulta.

RAZÓN 2 • PRECISIÓN

Se recupera por pedazos

El embedding de un documento entero promedia TODOS sus temas y no queda cerca de ninguno. El de un párrafo enfocado sí matchea una pregunta concreta.

Definición para el resto del semestre: el CHUNK es la unidad de recuperación. La búsqueda no encuentra "documentos" — encuentra chunks. Si la respuesta a una pregunta quedó partida entre dos chunks, ningún retrieval perfecto la va a traer completa.

Analogía: nadie encuentra nada en una biblioteca donde cada libro es UNA página gigante — ni en una donde cada página está cortada en confeti. El chunking es decidir el tamaño de página de su biblioteca.

Cuatro estrategias, de simple a sofisticada.

FIXED-SIZE

Cada N caracteres

Cortar cada N caracteres o tokens, con solapamiento. Ignora la estructura del texto — puede partir una frase en dos — pero es trivial y sorprendentemente competitiva. La del lab de hoy.

RECURSIVE

Respetando separadores

Intenta cortar por párrafos; si el pedazo sigue grande, por oraciones; luego por palabras. Respeta la estructura natural. El default sensato de los frameworks.

SEMANTIC

Por cambio de tema

Usa embeddings para detectar dónde el texto cambia de tema y corta ahí. Chunks temáticamente coherentes, a mayor costo de indexación.

HIERARCHICAL

Padres e hijos

Chunks pequeños para BUSCAR con precisión, ligados a su sección padre para ENTREGAR contexto completo al modelo. Lo mejor de ambos tamaños.

El MISMO texto, cuatro formas de partirlo

el chunk es la unidad de recuperación: donde cortes decide qué puede encontrar tu RAG

1 · FIXED-SIZE

corta cada N caracteres, caiga donde caiga ✕

REGLAMENTO DE TAREAS

§1 Las tareas valen máximo el 15% de la nota.
Deben publicarse con 3 días de anticipación.

§2 Cada día de retraso descuenta el 10% de la nota, hasta un máximo de tres días.

§3 El uso de IA debe declararse al entregar.
La política de integridad rige desde 2026.

⚠️ partió la idea del §2 en dos chunks:
ya no se puede recuperar completa

2 · RECURSIVE

intenta cortar por párrafos; si no cabe, por oraciones

REGLAMENTO DE TAREAS

§1 Las tareas valen máximo el 15% de la nota.
Deben publicarse con 3 días de anticipación.

§2 Cada día de retraso descuenta el 10% de la nota, hasta un máximo de tres días.

§3 El uso de IA debe declararse al entregar.
La política de integridad rige desde 2026.

✓ cada § queda entero: respeta las costuras del texto
(el default sensato de los frameworks)

3 · SEMANTIC

corta donde los embeddings detectan cambio de TEMA

REGLAMENTO DE TAREAS

§1 Las tareas valen máximo el 15% de la nota.
Deben publicarse con 3 días de anticipación.

§2 Cada día de retraso descuenta el 10% de la nota, hasta un máximo de tres días.

§3 El uso de IA debe declararse al entregar.
La política de integridad rige desde 2026.

tema: notas y entregas

tema: uso de IA

✓ agrupa §1 y §2 porque hablan de lo mismo:
chunks coherentes, a mayor costo de indexación

4 · HIERARCHICAL

hijos pequeños para BUSCAR, padre completo para ENTREGAR



la búsqueda hace match con un HIJO pequeño y preciso...
...pero al prompt se entrega el PADRE completo

Tamaño y overlap: no hay número mágico.

Chunks pequeños (~100–200 tokens)

✓ búsqueda precisa: el vector representa UNA idea

✗ contexto insuficiente: la respuesta llega recortada

Chunks grandes (~500–1000 tokens)

✓ contexto completo para el modelo

✗ vector difuso que matchea peor + ruido en el prompt

Overlap (~10–20%)

✓ una idea partida por el corte sobrevive en el chunk vecino

✗ algo de redundancia en el índice — precio barato

¿Cómo se elige entonces? Como todo en este curso: se MIDE. Dos configuraciones de chunking + su harness de M2 = la respuesta para SU corpus. Punto de partida razonable: 300–500 tokens con 10–15% de overlap.

Buscar por significado.

La pieza que ya conocen (S05, S06) — ahora como motor de búsqueda.

Ya los usaron dos veces. Hoy, un tercer papel.

En S05 los usaron para **comparar** (BERTScore rescataba la paráfrasis). En S06, para **evaluar** (dimensión 1 del harness). Hoy, el mismo objeto para **buscar**: la pregunta se convierte en vector, y se buscan los chunks cuyos vectores estén más cerca.

$$\text{similitud}(\text{pregunta}, \text{chunk}) = \cos(\theta) = (A \cdot B) / (\|A\| \|B\|) \rightarrow \text{top-k}$$

- **Por eso encuentra lo que el keyword search no ve:** "¿puedo viajar con mi bulldog en bodega?" matchea el chunk de "razas braquicefálicas", sin compartir ni una palabra.
- **Y por eso también trae ruido:** lo temáticamente cercano puntúa alto aunque no responda la pregunta — el mismo talón de Aquiles de S05.

Mismo modelo del harness: paraphrase-multilingual-MiniLM-L12-v2 — multilingüe, pequeño, y ya lo tienen descargado.

La base vectorial: qué agrega sobre una lista.

Con 50 chunks, un arreglo de numpy y un for alcanzan. La base vectorial existe para cuando eso deja de alcanzar:

ESCALA

Índices ANN

Con millones de vectores no se compara contra todos: índices aproximados (HNSW) encuentran los vecinos en milisegundos.

METADATOS

Filtrar + buscar

"Solo documentos de 2026", "solo la fuente oficial": filtro por metadatos + similitud, en la misma consulta.

PERSISTENCIA

Indexar una vez

El índice sobrevive al notebook: se construye una vez y las consultas siguientes no re-embeben el corpus.



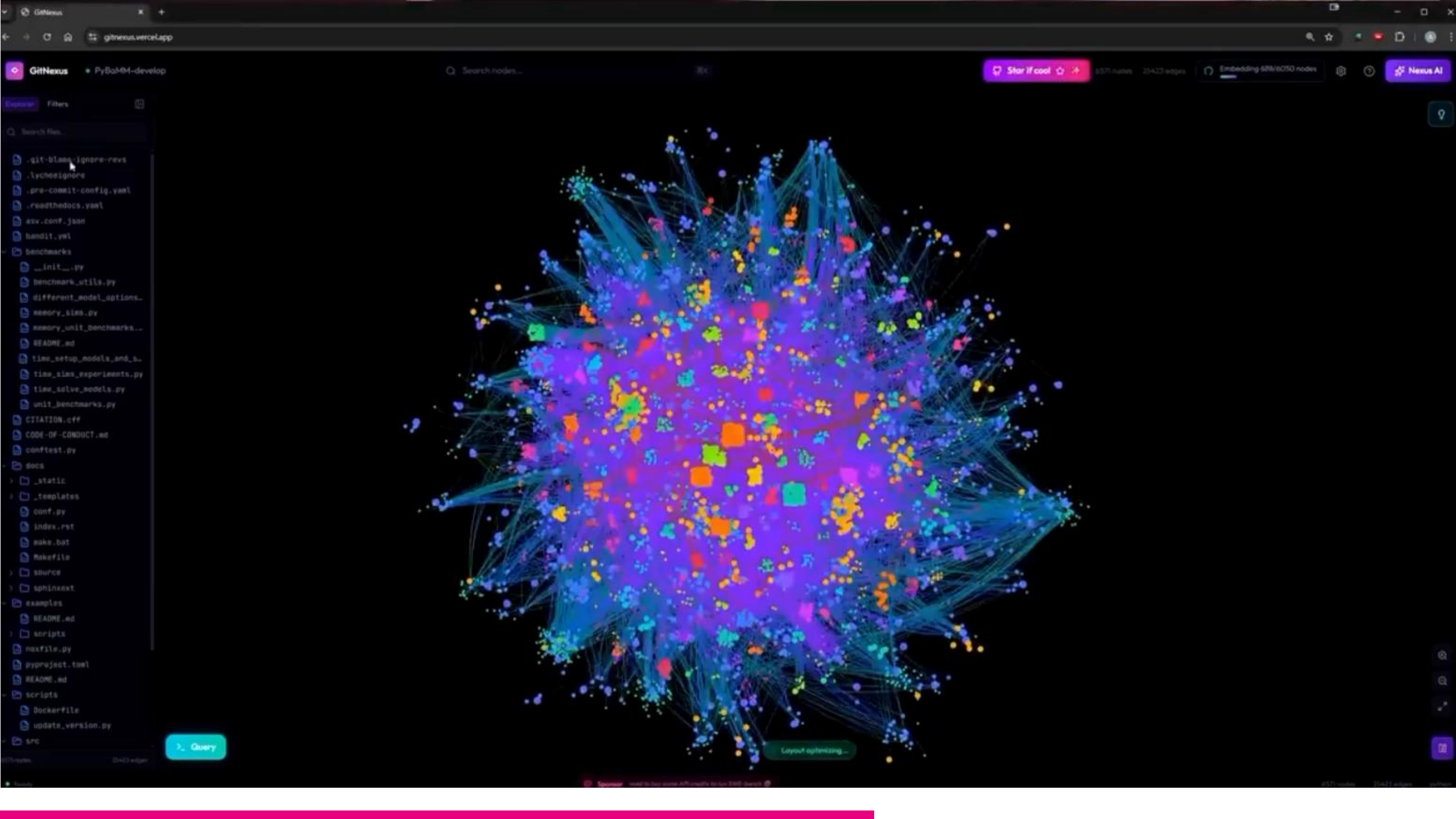
Chroma el default del curso: embebida, cero configuración, corre en Colab



FAISS la librería de Meta: máximo control y velocidad, más manual



Qdrant servidor dedicado: para producción, con API y filtrado rico



AI notebook — Lab A

Indexen su corpus y búsquenlo

Cargan documentos de su dominio, los parten con chunking fixed-size con overlap, los embeben y los guardan en Chroma. Luego consultan: comparan lo que encuentra la búsqueda por palabras clave contra la búsqueda semántica, con las mismas preguntas — incluidas dos donde el keyword search falla y una donde el semántico trae ruido.

`chunking → embeddings → chroma → top-k` • `keyword vs semántica` → `S07_Lab_RAG_ingenuo.ipynb`

Qué acaban de ver.

- **La semántica encuentra por significado** — la consulta coloquial llegó al chunk técnico correcto sin compartir palabras. Eso el keyword search no lo puede hacer.
- **Pero el top-k también trae ruido** — chunks del tema, que no responden LA pregunta. El retrieval ordena por cercanía, no por "responde/no responde".
- **Y el chunking decide qué existe** — si la respuesta quedó partida entre dos chunks, ninguna búsqueda la trae completa. El corte es una decisión de diseño, no un detalle.
- **Guarden sus consultas de prueba** — las que fallaron son candidatas perfectas para su eval set: casos difíciles reales de SU corpus.

Ingenuo, pero suyo.

Retrieve + augment + generate sobre su corpus. "Ingenuo" no es un insulto: es el baseline de M3.

Qué es (y por qué empezar ahí).

```
sistema_rag(pregunta) = generar( prompt(instrucción + top_k(pregunta) + pregunta) )
```

"Ingenuo" (naive RAG) significa: una sola búsqueda, un solo intento, sin trucos — sin hybrid search, sin reranking, sin transformar la consulta. Es deliberado: **primero el sistema más simple que funciona, medido; después, cada mejora tiene que ganarse su lugar contra él.** ¿Les suena? Es la misma lógica del baseline de M1 y M2 — es la tercera vez que este curso les hace el mismo movimiento, y no es casualidad.

La semana entrante (S08) le agregan técnicas avanzadas — hybrid search, reranking, query transformation — y CADA una se justifica con el delta que produzca en su harness. Sin RAG ingenuo medido, no hay delta que mostrar.

Los tres modos de fallo de un RAG.

RETRIEVAL

No encontró

El chunk con la respuesta no llegó al top-k: mala consulta, mal chunking o el documento no estaba. El modelo sintetiza sobre evidencia equivocada.

Se diagnostica mirando QUÉ trajo la búsqueda.

GENERACIÓN

Encontró pero ignoró

El contexto correcto estaba en el prompt y el modelo respondió de memoria igual — o mezcló contexto con memoria.

Se diagnostica comparando la respuesta contra el contexto entregado.

CORPUS

No estaba

La respuesta no existe en sus documentos. El sistema correcto dice "no está en mis fuentes" — el incorrecto inventa.

Se diagnostica con sus casos adversariales de S06.

Por eso "el RAG no funciona" nunca es un diagnóstico: hay que separar retrieval, generación y corpus. En S10, RAGAS les da métricas por componente (faithfulness, context precision/recall) — hoy, el diagnóstico es a ojo y con su harness.

Al notebook — Lab B

Armen el RAG ingenuo, de punta a punta

Conectan el retrieval del Lab A con el generador: prompt aumentado (con válvula de escape) → respuesta. Prueban las mismas preguntas de su eval set con y sin RAG, lado a lado. Y le hacen la prueba adversarial: una pregunta cuya respuesta NO está en el corpus — ¿dice "no está en mis fuentes" o inventa?

```
sistema_rag(pregunta) → respuesta con fuentes • con-RAG vs sin-RAG → S07_Lab_RAG_ingenuo.ipynb
```

Lo que acaban de construir.

- **Un RAG completo y sin frameworks** — siete etapas que ustedes pueden explicar línea por línea. Cuando adopten LangChain o LlamaIndex, sabrán qué hace cada caja.
- **La diferencia se ve a ojo** — en preguntas de datos concretos del corpus, el con-RAG responde con lo que dice el documento; el sin-RAG generaliza o inventa.
- **La válvula de escape trabaja** — ante la pregunta sin respuesta en el corpus, el sistema honesto dice "no está en mis fuentes". Si el suyo inventó: revisen el prompt — y reporten el hallazgo.
- **"A ojo" ya no es su estándar** — desde M2, "se ve mejor" no basta. Lo que sigue es medirlo — y eso es exactamente la tarea.

La vara ya existe.

La tarea de la semana: pasar el RAG ingenuo por el harness de M2.

La tarea: midan lo que armaron hoy.

```
harness(eval_set, sistema_rag) → scorecard_rag.csv vs scorecard_baseline.csv
```

Mismo eval set, mismo harness, misma rúbrica del juez — solo cambia el sistema. Es la primera vez que su vara de M2 hace su verdadero trabajo: atribuir una mejora. Qué mirar en la comparación:

- **¿Subieron los aciertos de dominio?** Es donde RAG debería pagar: preguntas cuya respuesta vive en el corpus.
- **¿Qué pasó con los adversariales?** El de alucinación debería mejorar ("no está en mis fuentes"). Si empeoró algo: ¿el retrieval metió ruido?
- **¿Dónde NO mejoró?** Un caso que sigue fallando con el chunk correcto en el prompt es fallo de generación, no de búsqueda — anótenlo: eso pide las técnicas de S08.

Entregable de la semana (no calificable, pero insumo directo de M3-S10): scorecard_rag.csv + 3 líneas de lectura honesta en el repo.

Dónde va esto.

	S07 · HOY	RAG ingenuo funcionando + tarea: harness al RAG	el baseline de la unidad
	S08	hybrid search (BM25+denso) · reranking · query transformation	cada técnica se gana su lugar con el delta en su harness
	S09	parcial corto (M1+M2, 15%) + checkpoint público de proyectos	individual — repasen transformers, evaluación y esta clase
	S10	tool use y agentic + evaluación RAGAS	entrega M3 (15%): sistema RAG + evaluación RAGAS

Lo que traen la próxima semana.

1

Su corpus real indexado: 10–30 documentos del dominio (los del lab fueron la prueba; ahora, los de verdad — con fuente y fecha).

2

El scorecard del RAG ingenuo (la tarea) junto al del baseline, en el repo.

3

Una lista corta de consultas donde su retrieval falló — son el material de trabajo de S08: sobre ellas probaremos hybrid search y reranking.

Dudas de la semana: por el foro del curso o el canal del equipo — respondo ahí para que le sirva a todos.

Ya saben medir. Ahora su sistema sabe consultar.

Hoy su modelo dejó de responder de memoria: busca, lee y cita. La próxima semana lo volvemos más listo para buscar — hybrid search, reranking, query transformation — y cada mejora tendrá que demostrarse contra el scorecard que van a correr esta semana.